LABSHEET=06 PRIYANSHU SHARMA CU24025O980 SECTION=(B) (47) Mastery in Data Analytics and Visualizations and Career Advancement (Technical)

```
# EXPERIMENT NO. 6
# PREDICTIVE ANALYTICS USING LINEAR REGRESSION
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
```

```
# ============================================================
# 1. LOAD DATASET
# ============================================================

file_name = "/content/house_price_prediction.csv"

df = pd.read_csv(file_name)

print("\n" + "=" * 60)
print("DATASET")
print("=" * 60)
print("priyanshu sharma")

print(df.head())
```

```
============================================================
DATASET
============================================================
priyanshu sharma
   area  bedrooms  bathrooms  stories  parking    price
0   850         2          1        1        0  3200000
1  1000         2          1        1        1  3600000
2  1200         3          2        1        1  4300000
3  1350         3          2        2        1  4800000
4  1500         3          2        2        1  5200000
```

```
# 2. DATASET INFORMATION
# ============================================================

print("\n" + "=" * 60)
print("DATASET INFORMATION")
print("=" * 60)

print("Rows:", df.shape[0])
print("Columns:", df.shape[1])

print("\nColumn Names:")
print(df.columns.tolist())

print("\nData Types:")
print(df.dtypes)
```

```
============================================================
DATASET INFORMATION
============================================================
Rows: 30
Columns: 6

Column Names:
['area', 'bedrooms', 'bathrooms', 'stories', 'parking', 'price']

Data Types:
area         int64
bedrooms     int64
bathrooms    int64
stories      int64
parking      int64
```

```
price        int64
dtype: object
```

```
# 3. CHECK MISSING VALUES
# ============================================================

print("\n" + "=" * 60)
print("MISSING VALUES")
print("=" * 60)

print(df.isnull().sum())
```

```
============================================================
MISSING VALUES
============================================================
area         0
bedrooms     0
bathrooms    0
stories      0
parking      0
price        0
dtype: int64
```

```
# 4. HANDLE MISSING NUMERIC VALUES
# ============================================================

numeric_columns = df.select_dtypes(include=np.number).columns

for column in numeric_columns:
    df[column] = df[column].fillna(df[column].mean())

print("\nMissing numeric values handled successfully.")
```

```
Missing numeric values handled successfully.
```

```
# 5. FIND NUMERIC COLUMNS
# ============================================================

numeric_df = df.select_dtypes(include=np.number)

print("\n" + "=" * 60)
print("NUMERIC COLUMNS")
print("=" * 60)

print(numeric_df.columns.tolist())
```

```
============================================================
NUMERIC COLUMNS
============================================================
['area', 'bedrooms', 'bathrooms', 'stories', 'parking', 'price']
```

```
# 6. SELECT TARGET COLUMN
# ============================================================

# Common target names for house price datasets
possible_targets = [
    "Price",
    "price",
    "SalePrice",
    "sale_price",
    "HousePrice",
    "house_price",
    "House_Price",
    "house_price_prediction"
]

target_column = None

for column in possible_targets:
    if column in df.columns:
        target_column = column
        break

# If common target name is not found,
# use the LAST numeric column as target
if target_column is None:
```

```
    if len(numeric_columns) < 2:
        print("\nERROR: Dataset must contain at least")
        print("one feature column and one target column.")
        exit()

    target_column = numeric_columns[-1]

print("\n" + "=" * 60)
print("TARGET COLUMN")
print("=" * 60)

print("Target Column:", target_column)
```

```
============================================================
TARGET COLUMN
============================================================
Target Column: price
```

```
# 7. SELECT FEATURES
# ============================================================

X = numeric_df.drop(columns=[target_column])
y = numeric_df[target_column]

print("\n" + "=" * 60)
print("INDEPENDENT AND DEPENDENT VARIABLES")
print("=" * 60)

print("Independent Variables:")
print(X.columns.tolist())

print("\nDependent Variable:")
print(target_column)
```

```
============================================================
INDEPENDENT AND DEPENDENT VARIABLES
============================================================
Independent Variables:
['area', 'bedrooms', 'bathrooms', 'stories', 'parking']

Dependent Variable:
price
```

```
# 8. CHECK FEATURES
# ============================================================

if X.shape[1] == 0:

    print("\nERROR: No numeric independent variables found.")

    exit()
```

```
# 9. TRAIN-TEST SPLIT
# ============================================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("\n" + "=" * 60)
print("TRAIN-TEST SPLIT")
print("=" * 60)

print("Training Data:", X_train.shape[0])
print("Testing Data :", X_test.shape[0])
```

```
============================================================
TRAIN-TEST SPLIT
============================================================
Training Data: 24
Testing Data : 6
```

```
# 10. CREATE LINEAR REGRESSION MODEL
# ============================================================

model = LinearRegression()

model.fit(X_train, y_train)

print("\nLinear Regression Model trained successfully!")
```

```
Linear Regression Model trained successfully!
```

```
# 11. MAKE PREDICTIONS
# ============================================================

y_pred = model.predict(X_test)
```

```
# 12. ACTUAL VS PREDICTED VALUES
# ============================================================

result = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": y_pred
})

print("\n" + "=" * 60)
print("ACTUAL VS PREDICTED VALUES")
print("=" * 60)
print("priyanshu sharma")

print(result.to_string(index=False))
```

```
============================================================
ACTUAL VS PREDICTED VALUES
============================================================
priyanshu sharma
 Actual    Predicted
7900000 7.772653e+06
6100000 6.030330e+06
5300000 5.168831e+06
7500000 7.593193e+06
7800000 7.952114e+06
9000000 9.028877e+06
```

```
# 13. PERFORMANCE METRICS
# ============================================================

mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

print("\n" + "=" * 60)
print("MODEL PERFORMANCE")
print("=" * 60)

print("Mean Absolute Error (MAE) :", round(mae, 4))
print("Mean Squared Error (MSE)  :", round(mse, 4))
print("Root Mean Squared Error   :", round(rmse, 4))
print("R² Score                  :", round(r2, 4))
```

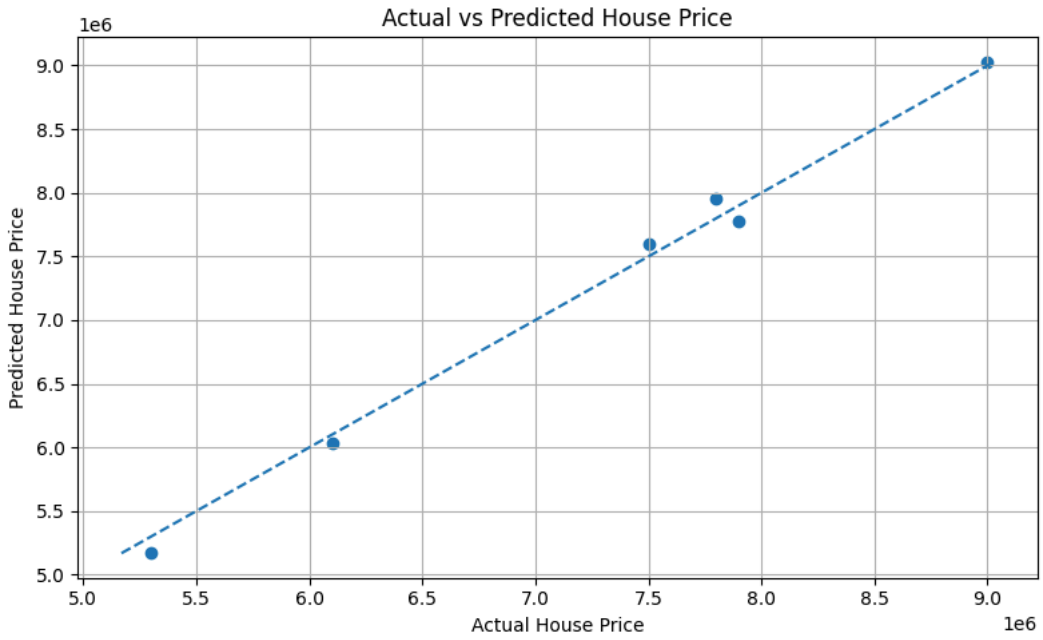
```
============================================================
MODEL PERFORMANCE
============================================================
Mean Absolute Error (MAE) : 100394.8975
Mean Squared Error (MSE)  : 11822309017.4954
Root Mean Squared Error   : 108730.442
R² Score                  : 0.9921
```

```
# 14. REGRESSION COEFFICIENTS
# ============================================================

print("\n" + "=" * 60)
```

```
print("REGRESSION COEFFICIENTS")
print("=" * 60)

print("Intercept:", round(model.intercept_, 4))

for feature, coefficient in zip(X.columns, model.coef_):

    print(
        feature,
        "=",
        round(coefficient, 4)
    )
```

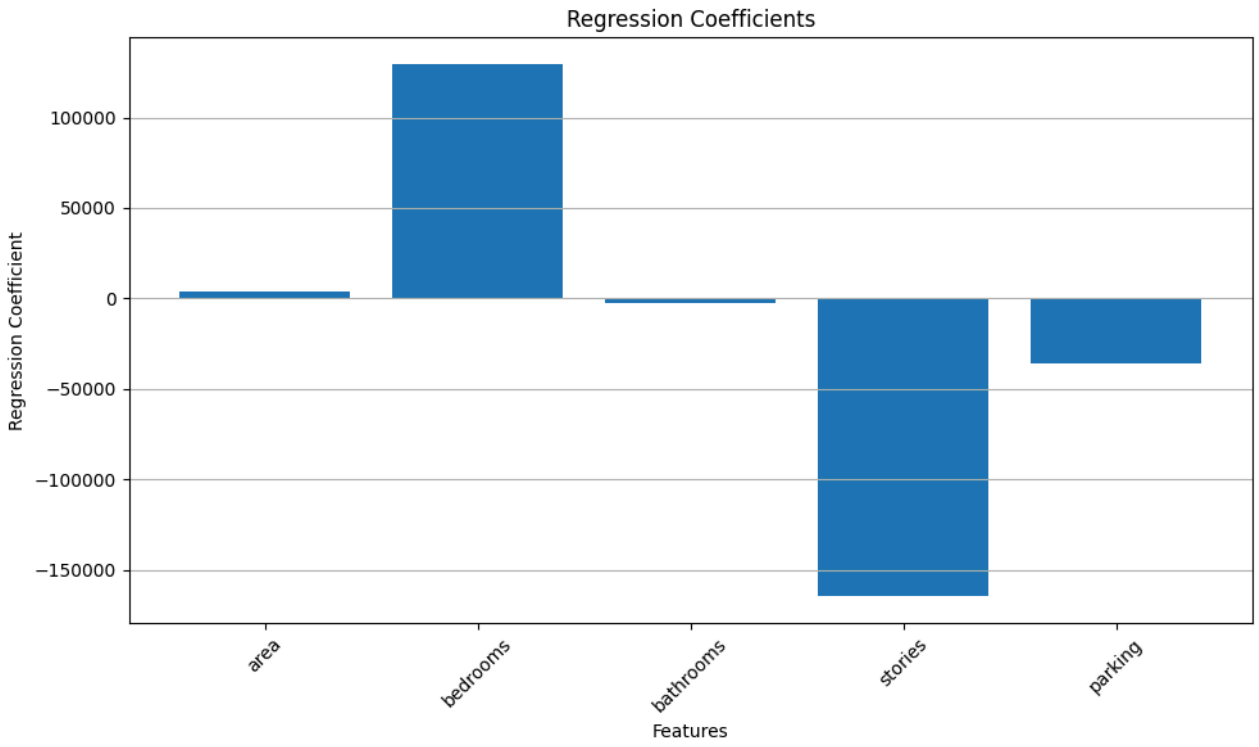
```
============================================================
REGRESSION COEFFICIENTS
============================================================
Intercept: -54274.8578
area = 3589.2101
bedrooms = 129445.0003
bathrooms = -2266.2245
stories = -164624.1979
parking = -35803.3293
```

```
# 15. ACTUAL VS PREDICTED GRAPH
# ============================================================

plt.figure(figsize=(8, 5))

plt.scatter(y_test, y_pred)

plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")

plt.title("Actual vs Predicted House Price")

minimum = min(y_test.min(), y_pred.min())
maximum = max(y_test.max(), y_pred.max())

plt.plot(
    [minimum, maximum],
    [minimum, maximum],
    linestyle="--"
)

plt.grid(True)

plt.tight_layout()

plt.show()
```




```
# 16. REGRESSION COEFFICIENT GRAPH
# ============================================================

plt.figure(figsize=(10, 6))
```

```
plt.bar(
    X.columns,
    model.coef_
)

plt.xlabel("Features")

plt.ylabel("Regression Coefficient")

plt.title("Regression Coefficients")

plt.xticks(rotation=45)

plt.grid(axis="y")

plt.tight_layout()

plt.show()
```




```
# 17. FINAL RESULT
# ============================================================

print("\n" + "=" * 60)
print("FINAL RESULT")
print("=" * 60)

print("Dataset:", file_name)

print("Target:", target_column)

print("\nLinear Regression completed successfully.")

print("\nPerformance:")
print("MAE  =", round(mae, 4))
print("MSE  =", round(mse, 4))
print("RMSE =", round(rmse, 4))
print("R²   =", round(r2, 4))

if r2 >= 0.80:

    print("\nModel Performance: GOOD")

elif r2 >= 0.50:

    print("\nModel Performance: MODERATE")

else:
```

```
    print("\nModel Performance: LOW")
    print("priyanshu sharma")

print("\nExperiment completed successfully!")
```

```
============================================================
FINAL RESULT
============================================================
Dataset: /content/house_price_prediction.csv
Target: price

Linear Regression completed successfully.

Performance:
MAE  = 100394.8975
MSE  = 11822309017.4954
RMSE = 108730.442
R²   = 0.9921

Model Performance: GOOD

Experiment completed successfully!
```